Avail: Grow Beyond One Chain

Version: 2.2

Published 12/06/2026

By Avail Team

Abstract

The introduction of off-chain scaling solutions has reshaped the requirements for decentralized blockchain infrastructure. Execution throughput has increased dramatically and helped reduce transaction costs. These improvements have come at the cost of further fragmentation of users and liquidity. However, they have also established core principles for amplifying the benefits of off-chain scaling, while addressing shortcomings of previous blockchain architectures. Avail approaches this new paradigm from first principles, designing a network from the ground up that is backwards compatible with the rest of the ecosystem while powering new blockchain designs.

Rollups have the ability to scale execution to billions of users, but as rollups grow, ensuring that transaction data is available to the network, becomes the bottleneck. To remove this bottleneck, rollups require data availability blobspace that is efficient, decentralized, secure and affordable. Avail's unique foundation achieves this, providing data availability to any blockchain with a simple integration. Anyone can easily verify the data's availability using succinct validity proofs. Avail light clients run on devices such as a user's phone to independently verify validity proofs locally. With fast and efficient end user verification, this approach helps scale decentralized blockchains to support mainstream use cases.

While data availability (DA) helps scale blockchains, it alone does not help solve blockchain interoperability. Here, Avail provides a solution for blockchains irrespective of whether they have integrated with Avail DA. Avail Nexus helps facilitate the coordination of assets between chains, without assets leaving their native chain. End users no longer need to bridge assets and can instead use Nexus to transact across any integrated blockchain. With Avail Nexus implemented, the walls between blockchain ecosystems disappear as each blockchain can interoperate within one large, connected network.

1 Introduction

Avail can be used by developers to build scalable apps that expand beyond the chain they are deployed on.

- **Scaling with Avail DA:** A pluggable, decentralized and modular data availability layer that provides verifiable data availability guarantees for blockchains looking to launch and scale. It combines validity proofs and data availability sampling to achieve these goals.
- **Interoperability with Avail Nexus:** A chain abstraction layer that leverages ZK technology to pass messages between any blockchain. Assets remain on their native blockchains, improving the developer and user experience.

The next sections outline key concepts and current challenges facing the blockchain ecosystem that Avail addresses.

1.1 Blockchain Scalability

The way that blockchains are built and scaled has changed. Previously an entirely new layer 1 blockchain had to be built just to test out a feature, or a new approach to scaling. Now it's possible to simply deploy a rollup on existing infrastructure, dramatically lowering the effort required from developers to deploy blockchains.

An ecosystem of modular blockchain developers are taking this approach even further by breaking up the components of a traditional monolithic blockchain, and optimizing each one. With modular blockchains, developers can combine independently optimized components to build blockchains that scale beyond the technical limitations of monolithic chains.

1.1.1 Rollup Adoption

Rollups are the state-of-the-art mechanism to scale blockchains today. Ethereum adopted the rollup-centric roadmap in 2020 and hosts multiple Optimistic and ZK rollups in production. Other monolithic blockchains are realizing the need to scale via off-chain execution as their blockchains become more congested with usage.

Rollups help scale monolithic blockchains by performing execution off-chain, freeing up the monolithic blockchain from having to process the execution itself. Execution proofs are then sent to the base layer, which it secures. This can enable batches of multiple transactions to be processed in a single transaction on the base layer, making transaction costs far lower for rollup users.

Introducing rollups has had numerous impacts on the blockchain ecosystem, including:

- Easier to build new blockchains as rollups. Developers can simply run a sequencer, add any arbitrary business logic, deploy it, and begin acquiring users.
- More rollups are getting deployed.
- Rollups are acquiring a growing number of users.
- New Rollup-as-a-Service (RaaS) providers are making it even easier to deploy rollups, sometimes with just a single click.

All of these factors, as well as the ability for rollups to scale execution, suggest that rollups are here to stay.

1.1.2 Modular Ecosystem

Moving execution off the main chain has helped inspire a movement towards modular blockchains. A modular ecosystem has formed around the idea of deconstructing the traditional, monolithic blockchain stack to build more scalable blockchains.

By breaking down a monolithic blockchain into different components like execution, settlement, ordering and data availability, each component can be independently optimized. These 'building blocks' can then be brought together to produce more scalable blockchains and provide greater flexibility for blockchain developers.

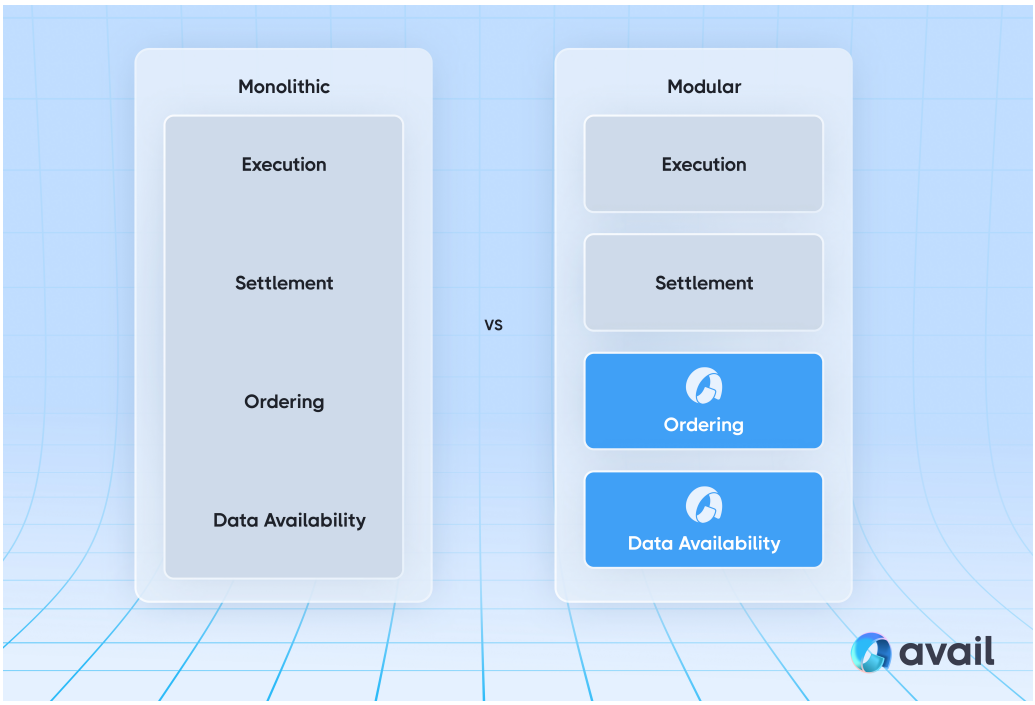


Fig. 1: Monolithic vs Modular Construction

The monolithic blockchain design by contrast is constrained by its requirement to handle multiple tasks within a single blockchain. Improving capacity in one part results in dependencies on other parts, which all must scale together. Leveraging the principle of the separation of concerns, modular blockchains enable each component to be independently optimized and used interchangeably with other components.

This has the added benefit of enabling developers to choose the best components for their individual use cases, and is similar to how modern infrastructure is built and scaled in web2.

1.1.3 Data Availability

Data Availability has become one of the most important parts of the modular blockchain stack to optimize for blockchains to scale. Transaction data that goes into updating new blocks needs to be kept secure and made publicly available to maintain the integrity of a blockchain. Without data availability guarantees, a user's funds can be confiscated or frozen without the user's consent, breaking the core promise of blockchain technology.

The introduction of rollups in the Ethereum ecosystem has significantly increased demand for data availability. The Ethereum roadmap has pivoted towards providing more data availability capacity to appease this demand. While Ethereum is the first monolithic blockchain to move in this direction, other monolithic layer 1 blockchains will also need to increase their data availability capacity as they begin to scale with rollups.

Every time a new block is added to a blockchain, nodes need to be certain that the transaction data is available. However, as the blockchain grows in popularity, so too does the amount of transaction data.

Taking a modular approach to scaling data availability provides superior scalability opportunities for blockchains. The data availability layer can be optimized to increase capacity in ways that are either difficult or impossible for monolithic blockchains to achieve.

In this paper, we will examine the design decisions behind Avail DA, culminating in a secure and decentralized data availability solution that scales with demand, thereby enhancing the viability of blockchains for emerging use cases, and improving the blockchain user experience.

1.2 Blockchain Interoperability

Historically, interoperability challenges have centered around interoperability between different layer 1 blockchains. With modular blockchains resulting in more blockchains being deployed, solving interoperability challenges has become even more pressing. Providing a secure and easy way to transact across multiple blockchains is important for the technology to succeed.

1.2.1 User Experience (UX)

Cross-chain interoperability is a UX issue that faces every blockchain ecosystem today. While some blockchains can provide a smooth UX within their ecosystem, users encounter friction as soon as they attempt to move their assets beyond it.

The evidence suggests there is strong user demand for moving assets across different blockchains. Billions of dollars worth of assets have been transferred between blockchains using cross-chain bridges, despite the inherent challenges they face. A cross-chain bridge typically involves locking up or burning tokens through a smart contract on the source chain, and unlocking or minting tokens via another smart contract on the destination chain.

Executing a cross-chain transaction with a bridge is not only cumbersome for users, but it also takes minutes to hours to transfer assets and costs tens of dollars. First, a user needs to find a bridge that supports both networks and is capable of transferring their desired assets. They then need to put their trust into that particular bridge, which can be challenging as bridges are repeatedly exploited by hackers. After identifying a suitable bridge, the user will likely need to adjust their wallet's network settings before signing multiple transactions to authorize the transfer and potentially incur high fees.

The end result is often confusing for new users, as they no longer have the original asset in hand. Instead, they now have a representation of the original token on the new network with a different name. This new asset is bound by the rules of the new network and may not allow the user to interact with dApps that the previous network could. If the user wants to go back to the original network, they would then need to repeat the process in reverse. Overall, bridging assets is a confusing, expensive, time-consuming and stressful process, even for seasoned blockchain users.

1.2.2 Developer Experience (DevX)

Building blockchain interoperability into an app can be more challenging. The cognitive load on developers is incredibly high as they need to navigate multiple hurdles like understanding different consensus mechanisms, state transition functions and the verification techniques of different blockchains.

To complicate things further, features, users and assets are all spread out across different blockchains, with no easy way for developers to connect them.

Cross-chain bridges cannot be a long-term solution, as the number of bridges required increases exponentially with every new blockchain that's released. Currently, the cross-chain developer experience is broken.

1.2.3 Application Specific Blockchains

Application specific blockchains are optimized to perform specific, or a group of related tasks. Examples of application specific blockchains include web3 payments, identity management and secure execution environments.

While many in the blockchain ecosystem are familiar with using smart contracts to build applications, a similar concept which instead uses purpose built, application specific blockchains is also gathering momentum. The enabler for this approach is seamless and reliable cross-chain interoperability.

When blockchains can be easily connected with each other, developers can combine application specific blockchains to create more feature rich and scalable applications. This has the potential to impact the usage and adoption of blockchain technology, similar to how microservices helped scale web2.

1.3 Decentralization

The decentralization of any blockchain will be determined by the least decentralized component in the stack. For example, many of today's rollups operate a centralized sequencer with plans to decentralize this over time. Avail provides the highest

decentralization guarantees of any modular data availability layer, so that developers have the most flexibility when implementing their blockchains.

Nominated Proof of Stake (NPoS) is one of the most decentralized PoS based blockchain designs available. NPoS irons out stake across the validator set so that a few nodes cannot easily control the network, as is the case with Delegated Proof of Stake (DPoS) based blockchains. Avail's blockchain implements NPoS, and is designed to support up to 1,000 active validators with relatively even stake shared between them.

Validity proofs from Avail DA can be quickly and easily verified by end users with light clients. The Avail light client is a small component used to help verify the data's integrity and availability. The Avail light client can be run in a variety of environments including on a browser, user's phone, smartwatch and Raspberry Pi. Light clients can provide millions of touchpoints in production, independently sampling and verifying data from the Avail network.

The Avail light clients also connect to form a P2P overlay network. This creates a replica of Avail's blockchain in the P2P network, providing additional decentralization guarantees and resilience. Avail DA is the only modular DA that can also sample from its P2P network.

1.4 The Avail Network

The Avail network in its entirety helps blockchains scale and connect with other ecosystems.

Avail DA helps blockchains scale by providing an abundance of data availability capacity. Its modular design scales data availability capacity with demand, and transaction data can be cryptographically verified quickly by anyone running an Avail light client.

Avail Nexus provides a hub for cross-chain transactions, where assets remain on native networks. Nexus facilitates interoperability between blockchains to provide a seamless cross-chain user experience.

Anyone can choose to leverage Avail in ways that compliment their particular needs. A developer looking for interoperability may just leverage the functionality offered by Nexus, one looking for scale may choose Avail DA or those looking for both can integrate Avail DA and Avail Nexus.

2 Avail DA

In this section, we discuss Avail DA, a general purpose DA layer that provides DA guarantees for other blockchains.

At a high level, a successful blockchain design needs to address the following components:

- **DA Layer:** This component receives transactional data and orders it without any execution. It then stores the data and ensures complete data availability in a decentralized manner.
- **Execution Layer:** The execution component takes ordered transactions from the DA layer and executes them. It creates a checkpoint/assertion/proof and submits it to the Settlement layer.
- **Settlement Layer:** This component represents the Verification/Dispute Resolution layer to which the system is anchored. The security of the design is dependent on the robustness and security properties of this component along with the DA layer. The checkpoint, or assertion, or proof submitted by the Execution layer is processed by this layer to guarantee that only valid state transitions are accepted in the system (provided that the data is available).

We envision multiple roll-up initiatives or legacy execution layers to form the Execution layer. The Settlement layer can be any secure settlement layer which supports the execution verification.

2.1 The Data Availablity Problem

Transactions on a blockchain are recorded in blocks, and the transaction data must be made available to network nodes whenever new blocks are added.

With this transaction data, full-nodes can re-execute the state of the blockchain and independently verify that state updates were done correctly. Without independent access to the transaction data, one cannot independently verify state updates, and instead must trust some other party that the blockchain is operating correctly. This breaks the fundamental promise of blockchain technology and can lead to vulnerabilities like having a user's assets lost or frozen.

Solving the data availability problem means addressing technical blockchain scalability constraints because more data availability capacity is required as the network grows. Simply forcing each node to keep all the data available leads to centralization over time, as only a few nodes can realistically afford to keep the data available over the long term.

Compressing transaction data is important, but there are limits to how far compression can help address the data availability problem.

Capacity is not the only concern either, the data availability problem is further complicated because it's a non-attributable fault. Identifying intentionally malicious actors and penalizing them accordingly in a decentralized network is difficult. Consider a case where a light client finds a block submitted by a full node that has missing data. The light client raises the alarm, but when other light clients go to query the full node, this time, the full node provides the data in full. In this scenario, other light clients can't be sure if it was the first light client that raised the alarm that was wrong, or if the full node is acting in bad faith. Left unaddressed, this can lead to critical vulnerabilities within the system that attackers can exploit.

Economic incentives also play a key role, as participants need to maintain the physical infrastructure required to run the data availability network. These incentives must be economically viable over the long-term.

Avail light clients gossip to create a P2P light client network, effectively replicating the Avail blockchain. This enhances network resilience, as light clients can sample data from both the server-based infrastructure and the P2P network. Verification is performed by the light client, which can be integrated into applications, providing end users with direct access to data availability verification.

2.2 Transaction Life Cycle

Avail DA is best thought of as a service that provides secure, verifiable data availability guarantees for other blockchains. It is an easily integrated component, requiring only a few lines of code.

Here is a brief run through of how Avail DA helps process a transaction:

- Transaction data gets sent to Avail DA and is indexed to an App ID. Although many chains use the same blobspace, nodes are not required to download data that is not relevant to them.
- The transaction data blob submitted to Avail DA is extended through erasure coding to add redundancy.
- Avail DA commits the data through KZG polynomial commitments to ensure the data has a footprint in the Avail block header.
- Blocks are then proposed and distributed to a decentralized network of validators to reach consensus and commit the next block to Avail's blockchain.
- Light clients can guarantee data availability immediately after finalizing, without waiting for a challenge period.

Although Avail light clients don't have a full record of the data themselves, they can query validity proofs via data availability sampling to provide mathematically verifiable guarantees that data is available. This is done without reliance on a full node and provides independent verification to any Avail light client.

Avail light clients then gossip to form a P2P light client network, which is a replica of the Avail blockchain. This adds resilience to the network as light clients can sample from both the server based infrastructure and the P2P light client network. Verification is done by the light client, which can be embedded into applications, bringing data availability verification directly to end users.

2.3 Using Avail DA

Avail DA can be integrated for any use case that requires a blockchain. As a general-purpose data availability layer, it allows developers to connect Avail DA with various execution environments and settlement layers, enabling the creation of fully functional general-purpose or application-specific blockchains.

Applications that handle a high volume of transactions or on-chain events and require a cost-effective, decentralized data availability solution are particularly well-suited for Avail DA. Examples include decentralized social media, DeFi, and web3 gaming, though Avail DA is not limited to these use cases.

We will explore the different ways Avail DA can be used to construct a blockchain below.

2.3.1 Validiums

Validiums are a type of blockchain construct that use validity proofs and publish transaction data to an external data availability provider, such as Avail DA. By publishing transaction data to an external source, the Validium construct inherits the integrity, security, and decentralization guarantees of the data availability provider.

Validiums then use a settlement layer, such as Ethereum. The Validium can send a unique blob reference of submitted data to the settlement layer. The rollup node can retrieve the data commitments from the settlement layer and access the transaction data from Avail to verify the validity of submitted transactions.

2.3.2 Volitions

Volitions are a scaling setup that enable two different transaction policies on the same scaling infrastructure. A volition can use both the Validium and Rollup policies to handle different transaction types. High volume data can use the Validium approach, while specific transaction types can use the Rollup structure.

2.3.3 Optimiums

Like Validiums, Optimiums use an external data availability provider and inherit its data availability, security and decentralization guarantees. They can be distinguished from Validiums because they use fraud proofs instead of validity proofs for verification.

2.3.4 Sovereign Rollups

Sovereign rollups using either ZK (validity proof based) or Optimistic (fraud proof based) constructs can also be integrated with Avail DA.

Sovereign rollups differ in the way they settle transactions. Instead of settling transactions on another blockchain, like Ethereum, they settle transactions locally. This removes the requirement for an external settlement layer. The sovereign rollup nodes instead settle transactions by verifying execution and data availability locally.

Avail DA can be integrated into a sovereign rollup relatively easily by embedding the Avail light client in the rollup node. The rollup node can then verify data availability independently and run on consumer grade hardware such as a user's phone.

2.3.5 BTC L2s

Developers can integrate Avail DA into BTC L2 constructs and benefit from interim finality from Avail DA before transactions are finalized on the Bitcoin network. Interim data availability guarantees can be achieved in around 40 seconds, and verified by end users using Avail's light client.

2.3.6 General Purpose Blockchains

Developers can use Avail DA to build general purpose blockchains. This can be achieved with one of the blockchain constructs mentioned earlier such as Validiums, Optimiums, Sovereign Rollups and BTC L2s.

2.3.7 Application Specific Blockchains

Application specific blockchains stand to benefit significantly from modular blockchain developments. They can be optimized to fulfill a specific role in the overall blockchain user experience, much like application specific microservices do for web2 today. For example, developers could leverage the full benefits of purpose built execution, settlement and data availability layers, to build a highly tuned blockchain for payments. This application specific blockchain could then be used by developers in their applications, much like Stripe is being used in web2 today.

For this to happen, cross-chain interoperability must be seamless, and Avail Nexus will play a key role in facilitating this.

As modular blockchain infrastructure and tooling matures, it appears that soon deploying an application specific blockchain will be as easy and inexpensive as it is to write and deploy a smart contract on a general purpose blockchain today. The exception being, it will be possible to avoid the scaling, performance and interoperability limitations that developers and users are confronted with currently on general purpose blockchains.

2.3.8 Utilizing Avail's Light Client

The Avail light client can generate equivalent DA guarantees to a full node and is light enough to run on consumer grade hardware. Light clients can also be integrated within blockchain nodes, providing interim finality guarantees. For example, a node could verify ZK proofs of execution and Avail's validity proofs for data availability to provide an interim finality guarantee prior to settling elsewhere.

2.4 Data Availability Guarantees on Ethereum

A settlement layer like Ethereum needs assurance that the underlying data is available when validating proofs and settling transactions.

To enable this, Avail guarantees data availability to a smart contract on Ethereum (or any settlement layer which supports this logic). This allows the checkpoint/assertion/proof accepting smart contract to get DA assurance directly.

A well-known approach for cross-chain message passing is through bridges. Using a bridge from Avail to the settlement layer, the Avail validators can attest to the fact that a particular piece of data is available. This approach assumes the super-majority of the Avail validator set to be honest and to keep the data available. For most applications, this is a good enough assumption.

Under this model, the rollup operator submits the data on Avail DA and gets back a finalized block, following transaction inclusion. A proof of transaction inclusion is computed and posted, along with proof of transactional validity, to the settlement rollup contract.

The bridge operator(s), post finalized block metadata on the settlement layer. The rollup contract, upon receiving the proofs, checks the correctness of the proofs against the bridge contract. The rollup transaction is accepted if the proofs are verified to be correct.

2.4.1 VectorX

VectorX has been primarily designed as a means to bridge AVAIL tokens to-and-from Ethereum. This can be expanded into other tokens, with other runtime upgrades, however, to allow for a generalized interface that can be expanded into different use-cases, an arbitrary message bridge on Ethereum is ideal. VectorX uses zkSNARKs to verify Avail's GRANDPA consensus.

- ZK light client secures Avail's data attestation bridge to Ethereum.
- A trust-minimized ZK bridge that provides Ethereum-settled rollups trust-minimized access to Avail's data attestations and brings Avail's DA securely to Ethereum.

2.5 System Goals

Avail DA needs to provide the following guarantees:

1) The decentralized data availability blockchain keeps producing a canonical chain of blocks even in the presence of an adversary that controls $<$ 1/3 of the validator nodes in the network.
2) The honest participants of the system with access to the canonical chain of block headers will not accept a block whose underlying data is unavailable, even if a very powerful adversary is controlling all the other nodes in the system. Under the assumption that there are enough honest participants, the data should remain available for a limited amount of time.

Avail DA achieves both these system goals, however we want to emphasize that both goals have different adversarial assumptions.

Goal 1 ensures the blockchain system continues to function in a decentralized manner as long as a super-majority of validator nodes remain honest.

Goal 2 ensures that an outside participant like an application running a light client, who has access to the canonical chain of block headers, need not trust any full node to have the guarantee that the underlying data for a particular block is available. This is an extremely strong assumption which eliminates any trust assumption to detect data hiding attempts.

2.6 Primitives

2.6.1 Why is redundancy important in ensuring Data Availability?

Suppose we have a block B divided into data chunks $D_1, \ldots, D_n$. The block producer wants to suppress one chunk. Without loss of generality, let us assume that the first chunk is hidden by the block producer. Clients can query one chunk at random to get guarantees that data is indeed available. It can repeat this process many times so as to get sufficient confidence that the data is accessible. Hence, for each query, the block producer needs to be lucky enough that D_1 is not queried.

However, with redundancy included by schemes like erasure coding, suppose the n chunks are encoded into $2n$ chunks. Erasure coding ensures that any n out of the $2n$ chunks are sufficient to recreate the data. This makes the hiding task harder for the block producer. To hide one particular chunk, it needs to make at least $n + 1$ chunks unavailable. As a client, querying a constant number of times gives a very high confidence that the data is indeed available. Hence, redundancy plays a vital role in the data availability. An erasure coding based design along with the related trade-offs are discussed in [1].

2.7 Kate Commitment Based Design

In this section, we first discuss polynomial commitments proposed by Kate *et al.* [2]. Then we go on to discuss a design based on Kate commitments as proposed in [3].

Given a polynomial $\phi(x) \in Z_p[x]$ over a bilinear pairing group, Kate *et al.* proposed a scheme to have a commitment to the polynomial using a single group element. Moreover, the scheme supports opening of a commitment at a point i to get $\phi(i)$ using a constant sized witness that allows a verifier to confirm that $\phi(x)$ was indeed evaluated at i to get $\phi(i)$. The commitment scheme is both computationally hiding and binding. The commitment scheme is additively homomorphic and supports a single witness for a batch of openings on multiple points of the same polynomial.

Given such a scheme, the block producer breaks the block data into chunks such that each chunk is an element of the field. It arranges the chunks into n rows and m columns such that it forms a $n \times m$ matrix D. It uses the evaluation form to construct a polynomial from each row to obtain $\phi_1(x), \ldots, \phi_n(x)$. It then commits each polynomial to get, $C_1, \ldots, C_n$ respectively. For redundancy, it extends $C_1, \ldots, C_n$ to $C_1, \ldots, C_{2n}$. It puts $C_1, \ldots, C_{2n}$ inside the block header and broadcasts it. Figure 2 shows the data arrangement and the corresponding commitments.

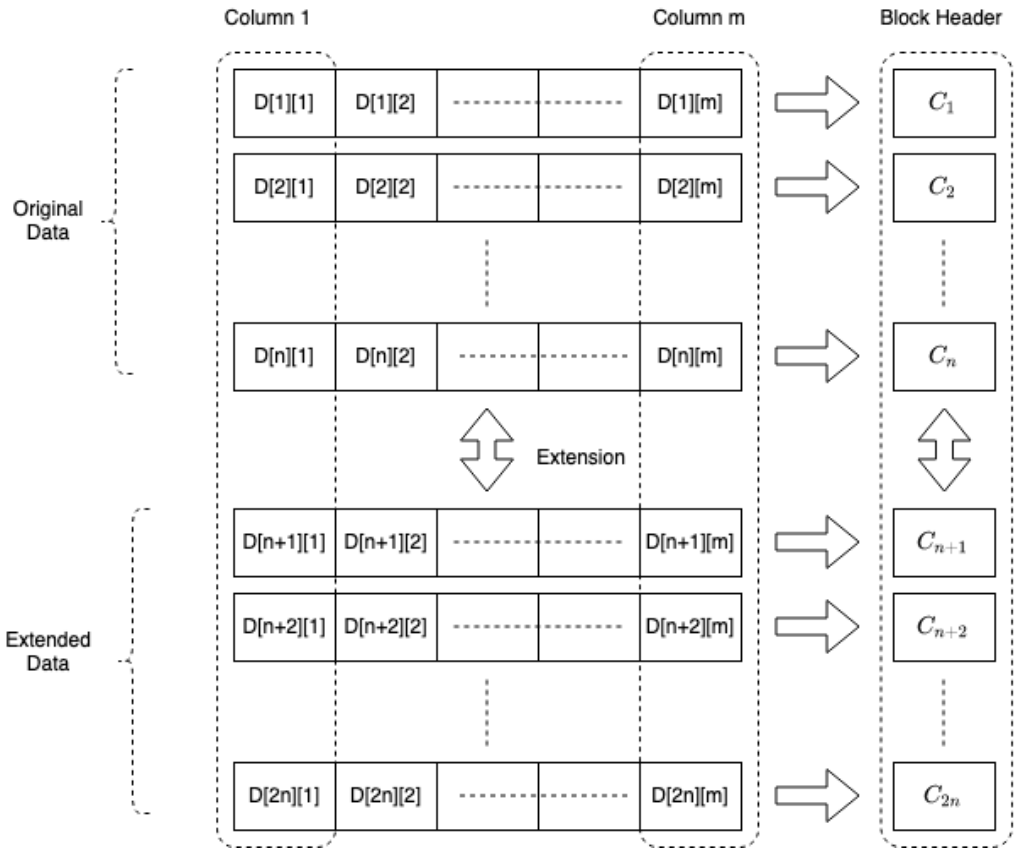


Fig. 2: Data Arrangement in Kate Commitment based DA

The light clients querying a data block will sample some chunk $D[i][j]$. Along with the data, the light client gets a witness $w[i][j]$ and it can immediately verify the validity using the Kate Commitment scheme discussed above. If it queries multiple chunks of the same row, the batch commitment scheme helps have a single witness for all the sampled points. Hence, the membership proofs are extremely efficient.

For full nodes, the system can have two types of full nodes:

1) Classical full nodes having entire blocks
2) Column full nodes, which keep only a single column of the data

For classical ones, it takes the entire matrix D, extends each column to $2n$ points and getting an extended matrix D'.

It then verifies for each row of D' whether the commitment to the ith row is C_i for $1 \leq i \leq 2n$.

For the column full nodes, they can fetch and keep only a column of the matrix D. They would extend each column to check whether they belong to the extended set of commitments. This is possible because of the homomorphic nature of the commitments and witnesses. In particular, having $D[1][j], \ldots, D[n][j]$ and $w[1][j], \ldots, w[n][j]$, the node can extend both to $2n$ points and immediately verify whether the extended openings are valid. If the number of such column full node is $O(m)$, and each column full node ensures at least one column is available, then collaboratively the entire data is available within the column full nodes.

As the block size grows, so does the number of commitments (keeping the number of columns fixed, the number of rows grow). Hence, the commitment size grows linear to that of the block size. However, we do not need any fraud proofs in such a system. For each light client, the communication and computational overhead is constant. For each full node, the computational overhead is $O(m * nlog(n))$ as it needs to perform $O(m)$ FFTs, $O(1)$ for each column, along with some pairing checks for verification. However, a column node has to only perform $O(nlog(n))$ work because it works on a single column of data.

2.8 Light Client

To guarantee data availability across a network, blockchains must either utilize full nodes, which download both the complete block data and block headers, or light clients that depend solely on block headers.

Light clients (LC) do not download the entire chain state but only the headers, which in turn introduces security assumptions between the LCs and full nodes. In the case of a potential dishonest majority of validators, those security assumptions can be broken. This is when Kate-Zaverucha-Goldberg (KZG) commitments and Data Availability Sampling (DAS) come into play.

On the Avail network, block data undergoes erasure coding before being secured with KZG commitments. Light clients in turn have to verify those commitments using random sampling of the committed data, in order to make sure that data is indeed available in the block.

Random sampling (DAS) is used as an efficient method for ensuring data availability without the need for exhaustive data retrieval. If random sampling returns positive results, i.e. block confidence above a certain threshold, the data in the block is said to be available, and Avail clients can rely on that availability. The application client, which is a part of the Avail Light Client, can then reconstruct the data it needs without having to download and verify the entire block.

We want our light clients to have very high data availability guarantees without hosting a full node or trusting any other peer. Keeping this in mind, we have developed a light client which keeps track of the chain header and performs availability queries (Data Availability Sampling) to gain confidence on the availability of interested blocks.

They also help contribute to data availability by hosting sampled data for other light clients within the P2P network. The client can gain as high confidence as deemed suitable for the specific application. We envisage the applications using Avail to host a light client.

2.8.1 P2P Light Client Network

Avail light clients connect to form a P2P light client network. With enough Avail light clients, the light client network begins to form a replica of the Avail blockchain, holding state and the last 256 blocks, the same as Full (non-archival) nodes.

Avail DA is the only DA layer to be able to sample from its network of light clients, providing additional redundancy to blockchains built on the DA layer. The Avail P2P light client network is constructed with the Kademlia DHT.

2.9 Application Specific Data Retrieval

We want our construction to allow applications to download data which is only relevant to them. To enable this, we want the full nodes to prove to an application client that the complete set of data relevant to the application has been communicated. To enable Application Specific Data Retrieval (ASDR), we have introduced two main fields:

- **AppID:** Every transaction has been associated with an application identifier (appID). Rollup or application specific chains can use an appID if they want to download parts of a block with only relevant application data. Any user is free to use the default appID ’0’ which is used for all types of transaction, including Avail system transactions. Any transaction with nonzero appID has to be of type application data submission only.
- **appDataLookup:** Every block header contains an index field, containing the starting cell indices of application cells inside the block.This is created by the block producer when creating the block by grouping the data matrix by appID and then flattening it to infer the starting position of each group.

The changes above together ensure that an application client can download only app-specific cells without downloading other application data.

However, it is worth noting that there is no reliance on the block producer to create the index or group the data correctly. This is because we create a deterministic algorithm that all honest application clients follow. Hence, all honest clients would retrieve the same app data for a particular appID. It can happen that clients get partial data, i.e. there exists data within the block associated with an appID which was not downloaded following the algorithm. However, the aim is to have the same view of their app to all honest app clients by providing them with the same data. The polynomial commitments inside the header assures that two clients with access to the same header cannot receive different sets of data.

2.10 Transaction Guarantees

There are different forms of guarantees users can receive per interaction based on the blockchain’s design.

- **Soft confirmation:** Centralized sequencer based rollups order the transactions and send soft confirmation to the user. Most users are satisfied because there is a negligible chance of reordering. Today, most rollups use this and explorers show this soft confirmation as transaction acceptance.

- **Subjective finality:** As soon as the ordered batch hits Avail and the block is finalized by the Avail validator set, the batch is essentially final. Any rollup node can then execute the batch to know whether the execution was done correctly, even if the proof of execution or fraud is not yet present. In based rollup constructions, where there is no sequencer for the rollup but the DA layer acts as the sequencer, the soft confirmations are unavailable and hence the users have to effectively wait for the subjective finality.
- **Objective finality:** It is impractical to assume that users are going to rerun batches of transactions to determine whether the execution was done correctly. Hence, most rollup constructions rely on validity or fraud proofs, which can be efficiently verified by the user. Once there is a validity proof which verifies a batch, the user can be assured of the correctness of execution by verifying it and hence getting objective finality. In optimistic constructions, only objective finality might be delayed because the user needs to wait for a challenge period, but with newer constructions where fraud proofs are also ZK proofs of incorrect transition, challenge periods can be further reduced.

3 Blockchain Network and Infrastructure

3.1 Types of Nodes

We consider the following types of nodes for Avail:

- **Full Nodes:** The full nodes download the blocks and validate their correctness but do not participate in consensus. They can choose to be an archive node and store the whole chain or be regular full nodes with block pruning (keeping only state and the last 256 blocks) but are not incentivized to participate or remain honest.
- **Validator Nodes:** The validator nodes take part in block generation and decide on transaction inclusion and ordering. These nodes are incentivized to participate in consensus and host the blockchain. Essentially, they are also full nodes with some stake in the system.
- **Light Clients:** These are clients with resource constraints who have access to only the block header and query transactional data from other full nodes on an as-needed basis. They want to have high confidence that the block is available, and when querying data they want a proof that the data belongs to the block.

Network participants are all incentivized to behave correctly. With NPoS, anyone can participate in ensuring the network remains stable. For this:

- Validators have their own stake and need to behave correctly to receive rewards and earn commission with their nominators. In case they produce a bad block or are not online, they will get ”Slashed” and a portion of their stake will be taken from them.
- Nominators are not direct network participants, but they nominate one or multiple validators to earn some rewards.
  - Nominators are incentivized to choose their validators correctly because in case of a slash, they will also lose a proportion of their fees.
  - All the selected validators will earn the same rewards meaning that, for decentralization purposes, nominators are incentivized to choose validators with less total stake (validator own stake + all nominators stake) to receive more reward, and hence balance the network stake amongst validators.
  - Nominators can also choose to stake via nomination pool, this is mostly used when the network staking is crowded and the minimum amount to stake is high. Pools can be proxied as team nominations, and even small amounts can be staked. The pool owner is in charge of choosing the validators and can perform various actions for the pool.

3.2 Consensus

We need our validators to reach consensus on the next block, which contains the ordered set of transactions along with redundant erasure coded data. Although there exists many possible blockchain consensus protocols, we opted for the Proof-of-Stake (PoS) family of consensus algorithms. We wanted a PoS consensus that supports high number of validators, provable finality and a robust security framework. In particular, we chose the BABE/GRANDPA hybrid consensus [4].

It uses two separate protocols for block production and finality. The block production is done using Blind Assignment for Blockchain Extension protocol (BABE) [5]. The block producers produce blocks based on a Verifiable Random Function (VRF). The protocol does not assume access to a central clock. In case no validator is chosen as the block producer for a particular slot, or if the chosen producer(s) go down, there is a secondary block producer who can step in. BABE ensures liveness, which guarantees that transactions submitted to honest players will eventually be inserted into the chain.

GRANDPA (GHOST-based Recursive ANcestor Deriving Prefix Agreement) [6] is a finality gadget ensuring provable finality. Without the finality gadget, the users can only have probabilistic finality, like in classical blockchain systems. GRANDPA guarantees that blocks reach quicker finality, and a finalized block can never be reverted.

We have chosen a 20-second block time and with that we achieve a 40-60 sec DA guarantee.

3.3 Full Nodes

The full nodes are not validator nodes in the Avail network, but instead participate in ensuring the network is stable and working as intended. Our design is a standalone blockchain, built using the Substrate framework.

1) Substrate provides great flexibility to implement custom runtime logic with inherent BABE/GRANDPA consensus support. We refactored some major components, like the system pallet.
2) The major changes that we made are:
   - The contents of the block are arranged into the data matrix, extended and polynomial commitment generation is performed during the block building process.
   - The header structure is changed to contain the commitments as part of the header.
   - Block sizes are made dynamic, according to the number of transactions on the mempool. However, the maximum block size allowed is 2MB erasure coded to 4 MB. This limit can be increased and has been stress tested up to 128MB.
   - Additional RPC methods to support data availability queries are introduced. These can be used by light clients to gain confidence about availability and inclusion.

3.4 Analysis

3.4.1 Attack Vectors

Any DA layer needs to counter the following attacks on data availability:

1) A super-majority of the validators in the DA layer wants to change the ordering of an already finalized block.
2) A super-majority of the validators create a wrong block header, i.e. the commitment of the data present in the header is wrong.
3) A super-majority of the validators want to hide at least one chunk of the block. For a light client, this would mean not being able to detect hiding of data with a non-negligible probability. For a full node, this would mean not being able to reconstruct the data.

We take the case of the super-majority of validators attacking the DA layer because we want to minimize the honesty assumption required for our design.

Under the assumption that finality is reached, the reordering attack (attack 1) is hard to mount. This is because, to successfully mount this attack the attacker would need to break the non-equivocation property of the underlying consensus. In particular, the finality layer ensures that if equivocation occurs, the dishonest parties can be identified and their stake slashed.

In the subsequent sections, we show how Attack 2 and Attack 3 are countered with Kate Commitments.

3.4.2 Kate Commitments

Attack 2 amounts to a wrong commitment by the block producer. Without loss of generality, suppose C_1 is wrong. This would mean that at least one out of $D[1][1]$ to $D[1][n]$ does not belong to C_1. Again, let us assume $D[1][1] \notin C_1$. This would mean that at least one of this is true: $D[n+1][1] \notin C_{n+1}, \ldots, D[2n][1] \notin C_{2n}$. This is because $C_{n+1}, \ldots, C_{2n}$ are extended from $C_1, \ldots, C_n$ and $D[n+1][1], \ldots, D[2n][1]$ are extended from $D[1][1], \ldots, D[n][1]$. Hence, such an attack is caught by a light client with overwhelming probability.

In case of attack 3, a light client querying constant samples can achieve arbitrarily high confidence that the data is indeed available. This is due to the redundancy in the data, as discussed previously. For a full node, it needs to download the at least n chunks from each column so that it can reconstruct the entire extended data. It checks the commitments to know whether the downloaded data is correct. A column full node needs to perform similar operations but only for one column of data. Even if the supermajority wants to suppress data after some time, the light client P2P layer contains the data with redundancy. This enables new clients to recover block data without reliance on the full nodes or validators.

Hence, this approach mitigates all discussed attacks and achieves the goals set in Section 2.5.

4 AVAIL Token

The AVAIL token is designed to play an essential role for the Avail network, enabling secure scaling and interoperability across different blockchains.

The AVAIL token has been designed with multiple levels of utility for accessing critical blockchain infrastructure from the Avail network. It includes a governance model that fosters collaboration, has a balanced distribution, and sustainable incentivization mechanisms. Together, the AVAIL token is set to empower and enable blockchain users from across the blockchain ecosystem, enhancing blockchain scalability and interoperability.

4.1 AVAIL Token Utility

The AVAIL token will drive a circular economy within the Avail network:

- **Staking:** AVAIL token staking will secure Avail DA.
- **Transaction Fees:** DA transaction fees are paid in AVAIL. This helps ensure a self-sustaining network with incentives aligned across all participants.
- **Governance:** AVAIL holders can propose and will be able to vote on network upgrades, changes, and community initiatives, ensuring a decentralized governance structure.
- **Access to Services:** Tokens can be used to access services within the Avail ecosystem, such as data availability and soon, interoperability services.

5 Avail Nexus

Chain abstraction is the problem of letting a user spend value on chain B using assets that live on chain A, ideally in a single signature, with no manual claim step, no wrapped representation of the source asset, and no waiting for a bridge's challenge window. The user-side framing is well known. The protocol-side framing is less well agreed on, and that is what Nexus V2 takes a position on.

Bridges solve this with lock-and-mint: deposit on A, mint a wrapped token on B, redeem later. The cost is wrapped fragmentation, redemption rounds, and a per-bridge trust assumption. Classic intent protocols (CoW, UniswapX) solve it with off-chain solver competition: a user signs an intent, a solver fills it on-chain, and the chain itself is the settlement layer. The cost is that there is no shared sequencer across solvers, order matching, slashing, and competition design are each protocol's problem. Shared-sequencer rollups solve it by collapsing the participating chains into a common ordering layer, which is too tight a coupling for a protocol that wants to span chains it doesn't own.

Nexus V2 takes a different position. It is an intent protocol plus a lightweight L2-style state machine that uses Avail as its data availability and ordering layer. Users sign intents, solvers fill them in seconds against per-source escrow contracts, and a separate validator set re-executes the state machine from Avail and authorises settlements back to source chains. The novel surface is a canonical intent state machine that lives off the user's chain, with settlement instructions derived deterministically from data availability rather than from any single trusted party's claim.

The design goals are:

1) **Single-signature UX.** One signature from the user authorises the deposit and fulfilment of the request on every source chain involved; settlement is then authorised by the validator set, not by the user.
2) **Sub-block fulfilment latency.** A solver should fulfil before the source chains have finalised, fronting capital against on-chain confirmation rather than waiting.
3) **Source-chain-native settlement.** The user receives the destination asset itself, not a wrapped representation; the solver is reimbursed in the source token directly.
4) **Separable trust between liveness and safety.** The sequencer is trusted to *include* and *order* events. It is not trusted with state correctness, anyone who reads Avail can re-execute and detect divergence. Validators authorise settlement; the sequencer cannot.

The rest of this paper walks the protocol from intent to settlement. Sections 5.2 and 5.3 are load-bearing: the request type and the solver lifecycle are where most of the design choices live. Sections 5.4–5.6 cover the state machine, sequencing, and validator role. Section 5.7 covers the middleware, the client-facing layer that sits between the SDK and the statekeeper in production. Section 5.8 consolidates the trust model. Section 5.9 names the open questions.

For component-level architecture and on-the-box layout, see `docs/ARCHITECTURE.md`. For MPC ceremony details, see `docs/MPC.md`. For solver rebalancing operations, see `docs/REBALANCER.md`. This paper complements those documents; it does not reproduce them.

5.1 Protocol Overview

Four actors participate in every Nexus V2 transfer:

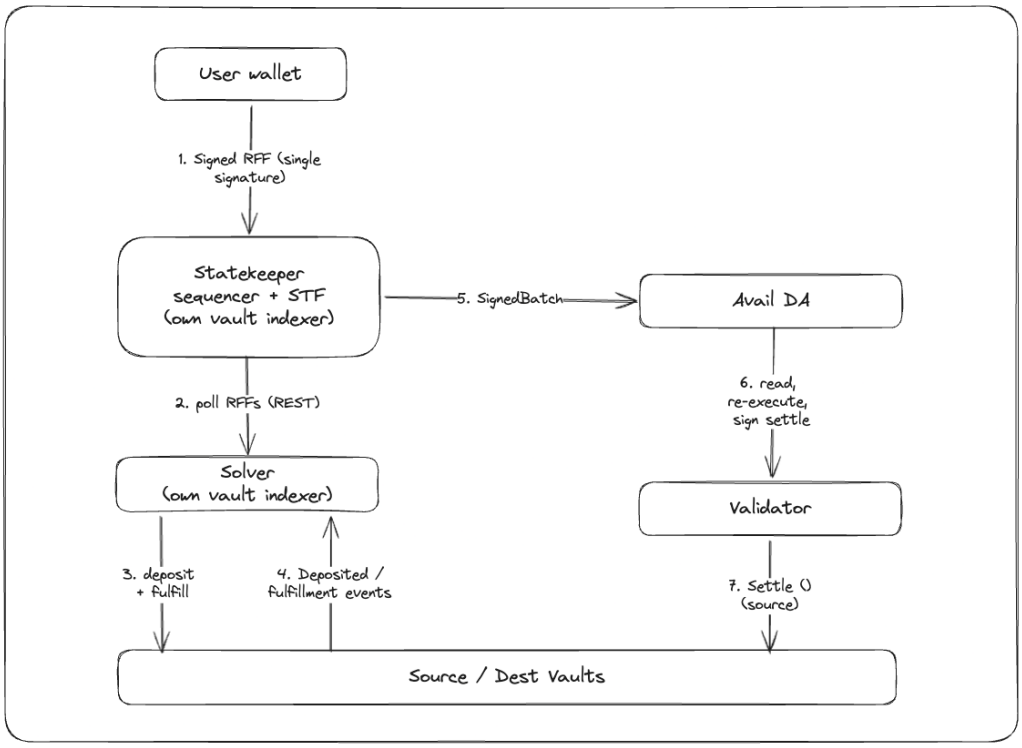


Fig. 3: Nexus V2 Protocol Overview

1) **The user** signs a Request for Funds (RFF) and submits it to the statekeeper. In production the submission goes through a stateless middleware gateway (see Section 5.7) (gasless approval sponsorship, balance aggregation, gas estimation) which forwards it to the statekeeper's REST API. The middleware is replaceable; the statekeeper API is the protocol's actual entry point.
2) **The solver polls the statekeeper** (`GET /rffs`) to discover new RFFs. The REST feed is a discovery channel only, the solver does not rely on the statekeeper's view for decisions about its own capital.
3) **The solver deposits, then fulfils.** It calls `Vault.deposit(...)` on each source chain (paying gas, earning the per-source fee), then `Vault.fulfil(...)` on the destination chain, fronting its own capital to deliver the user's tokens.
4) **The solver's own indexer confirms** the `Deposited` / `Fulfilment` events. It watches every Vault contract directly rather than trusting the statekeeper to have sequenced them, this is the evidence its capital decisions rest on.
5) **The statekeeper batches to Avail.** It merges two streams, API transactions (RFF creation, expiry) and chain events from its own indexers, executes them through a pure STF, and submits Ed25519-signed batches to Avail.
6) **The validator reads, re-executes, and signs settlement.** It fetches batches from Avail, verifies the sequencer signature, re-executes the STF, independently re-verifies the chain events against its own indexer, confirms the header's state root, and signs the settlement data.
7) **The validator submits `Vault.settle(...)`** on each source chain, reimbursing the solver from the escrowed source funds.

Two data planes underpin this:

- **Avail DA** carries the ordered, signed batch stream. It is the canonical source of "what happened, in what order." Anyone with Avail access can reconstruct the full state.
- **Vault contracts** on each chain carry value. They hold user deposits in escrow until either a fulfilment triggers settlement to the solver or an expiry triggers a refund to the user.

A sequencer tick is approximately 5 seconds. An Avail block is approximately 20 seconds. A single Avail submission can therefore carry several Nexus blocks; batches amortise DA cost across multiple sequencer ticks.

Actor	What they sign	Trusted for
User	RFF (canonical request hash, wrapped per universe)	Their own intent only
Solver	Source / destination chain transactions	Self-interest only, no protocol-level trust
Statekeeper	Each batch (Ed25519, registered pubkey)	Liveness and canonical ordering only
Validator	Settlement data (ECDSA over EIP-191)	Settlement authorisation and header finalisation

One design choice worth surfacing immediately, because the rest of the paper builds on it: **settlement is not an RFF status**. The RFF state machine has four states (Created, Deposited, Fulfilled, Expired). Settlement and refund are instructions emitted by the STF into the block header, not transitions in the RFF lifecycle. Section 5.4 returns to why.

5.2 The RFF Intent Model

The Request for Funds (RFF) is the entire user-facing surface. Everything else, solver economics, statekeeper sequencing, validator settlement, derives from how RFFs are shaped and signed.

5.2.1 The Request Struct

A `Request` is laid out to match the Solidity Vault contract one-to-one, so the same struct round-trips between Borsh-serialised Nexus state, ABI-encoded calldata, and event payloads:

```
struct Request {
    sources:              Vec<SourcePair>,     // 1..N source legs
    destination_universe: Universe,
    destination_chain_id: U256,
    recipient_address:    H256,                // 32-byte padded address
    destinations:         Vec<DestinationPair>,
    nonce:                U256,                // per-user replay protection
    expiry:               U256,                // unix seconds
    parties:              Vec<Party>,          // signers (typically 1: the user)
}
```

`SourcePair` carries `(universe, chain_id, contract_address, value, fee)`, the chain, the token, the amount the user puts in, and a **per-source fee** that the solver collects on that source. `DestinationPair` is `(contract_address, value)`, what the user wants delivered on the destination chain.

A handful of these field choices encode real design positions.

Per-source fees. Each source leg carries its own fee, denominated in that leg's source token. This attaches the fee to the leg that bears the solver's risk and gas cost. A multi-source RFF (say, USDC on Arbitrum plus USDT on Base) is naturally a collection of independently-priced legs from the solver's accounting standpoint, even though they share fulfilment on the destination.

Universe as a first-class field. `SourcePair.universe`, `destination_universe`, and each `Party.universe` carry the chain family explicitly (currently EVM; the enum allows for future families). This matters because the same logical address means different bytes on different VMs, and the signing scheme is universe-specific. Pushing the universe into the type forces every consumer (STF, vault encoder, signature verifier) to handle it explicitly rather than assume EVM.

`parties: Vec<Party>` rather than a single signer. Today every RFF has one party (the user). The vector exists for forward compatibility: co-signed RFFs (multisig users, delegated approvals, account abstraction with session keys) drop in without a struct change or a migration.

Nonce and expiry as `U256`. The Vault contract on each source chain reads these as `uint256`. Keeping the Rust types `U256` rather than `u64` avoids silent truncation at the ABI boundary and makes per-user nonce spaces effectively infinite.

5.2.2 Two Hashes: Identity vs. Signature

The RFF has two distinct hashes, and the distinction is load-bearing.

The **request hash** is the canonical identifier. It is `keccak256` over the ABI-encoded request fields and is universe-agnostic. The Vault contract uses it as the indexed value in `Deposit` and `Fulfilment` events; the STF uses it as the JMT key for RFF state; off-chain indexers, mempools, and the solver share the same identifier across every chain involved. One RFF, one hash, everywhere.

The **signing hash** is universe-specific. For EVM, the Vault contract wraps the request hash in a custom message format before applying the EIP-191 envelope:

```
SIGNATURE_PREFIX  = "Sign this intent to proceed \n"
msg               = SIGNATURE_PREFIX || hex_string(request_hash)
signing_hash      = keccak256( "\x19Ethereum Signed Message:\n" || len(msg) || msg )
```

Wrapping the request hash with a human-readable prefix means the user's wallet shows "Sign this intent to proceed" rather than an opaque 32-byte hex string. Wrapping the result with EIP-191 prevents the same signature from being valid as a transaction. The wrapper varies by universe (a future Solana variant would use that universe's domain separator); the identifier inside the wrapper does not.

This split is the protocol's main accommodation to heterogeneous signing. The mempool, STF, and indexers speak one identity. The user's wallet and the Vault contract speak the universe's local signature dialect.

5.2.3 Lifecycle

An RFF has four states, persisted by discriminant value (the values are frozen; reordering them is a migration):

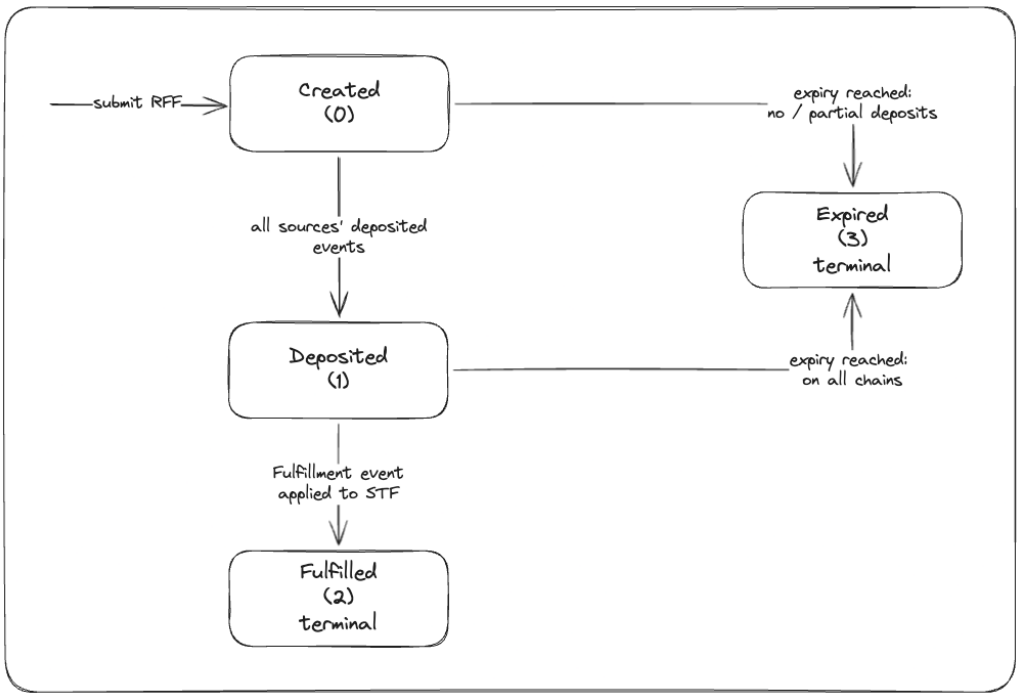


Fig. 4: RFF State Lifecycle

`Created → Deposited` requires *every* source's `Deposited` event to be applied to the STF. The transition is per-source-accumulating: the STF tracks a `deposited_sources: Vec<bool>` against the RFF leaf and only flips the status once all entries are true. Once `Deposited`, the RFF is eligible for fulfilment.

`Deposited → Fulfilled` is triggered by a `Fulfilment` event from the destination chain. The STF binds the solver's address into the RFF leaf at this point and, in the same step, generates a `SettlementInstruction` for every source. We come back to this in section 5.4; the important thing here is that "Fulfilled" is what the user cares about (their funds arrived), and settlement is what the solver and validators care about (the solver gets reimbursed). Decoupling them is deliberate.

`Created → Expired` and `Deposited → Expired` are both possible. Expiry is a sequencer-generated transaction validated against on-chain block timestamps; the STF requires both source and destination chain timestamps to have passed the RFF's expiry before accepting the transition. For an RFF that had any confirmed deposits, expiry generates *refund* settlement instructions (recipient = original user). For one with no confirmed deposits, expiry is a no-op cleanup.

Two implications follow from this design.

Multi-source atomicity is enforced by the STF, not by the contract. No source chain knows that an RFF has multiple sources. Each Vault contract just sees its own `deposit` and emits an event. The state machine is the single authority that aggregates these per-source events into a global "all deposited" condition.

Settlement is not a state. Refunds and reimbursements happen outside the four-state machine, via instructions in the block header. This keeps the user-visible state binary (did my bridge work?) while letting validator-side bookkeeping handle the long tail (which sources have been reimbursed, with what retries, by which validator). The next section uses this same separation to discuss why settlement-as-instruction matters.

5.3 Solver Lifecycle and Economics

The solver is the protocol's pricing function, its liquidity source, and the bearer of every operational risk that does not fall on the user. Its economic design is the most consequential set of choices in Nexus V2, and the one most divergent from comparable intent protocols.

5.3.1 The Solver as a Composition of Loops

A solver is six concurrent loops, each tied to a different external clock:

Component Role	File	Cadence
Poller Pull new and updated RFFs from statekeeper REST	`poller.rs`	1s
Depositor Submit `Vault.deposit(...)` on source chains	`depositor.rs`	1s
Fulfiller Submit `Vault.fulfil(...)` on destination chains	`fulfiller.rs`	1s
TxHandler Per-chain nonce manager, sliding window of in-flight txs	`tx_handler.rs`	event-driven
Indexer + event handler Independent chain watching, updates RFF store	`event_handler.rs`	2s confirmation
Rebalancer Bridge inventory between chains via Stargate / Across	`rebalancing.rs`	900s

The TxHandler is shared by depositor and fulfiller through cloneable `TxSubmitter` handles, one per chain. It owns the nonce, holds a configurable sliding window of in-flight transactions, and broadcasts `TxUpdate` (Submitted / SendFailed / Confirmed / Reverted) on a tokio broadcast channel. Depositor and fulfiller subscribe and filter updates by `TxId.kind` to handle only the ones that belong to them. This factoring matters more than it looks: nonce mismanagement is the most expensive class of solver bug, and isolating it behind one per-chain owner removes a category of race conditions that arise when independent loops both try to send transactions on the same chain.

The indexer is also separated. The solver runs its own EVM indexer that watches every Vault contract directly, with approximately a two-second confirmation latency. The statekeeper has its own indexer feeding the STF; the solver does not consume the statekeeper's view. This is a protocol-level decision, not an implementation accident: **the solver does not trust the statekeeper for decisions about its own capital.** When the solver decides whether a deposit it just made has been confirmed (and therefore whether it should proceed to fulfil), it consults its own indexer's evidence of the `Deposited` event. The statekeeper's view is for downstream consumers, not for the solver's own risk management.

5.3.2 The Five Phases of an RFF, From the Solver's Side

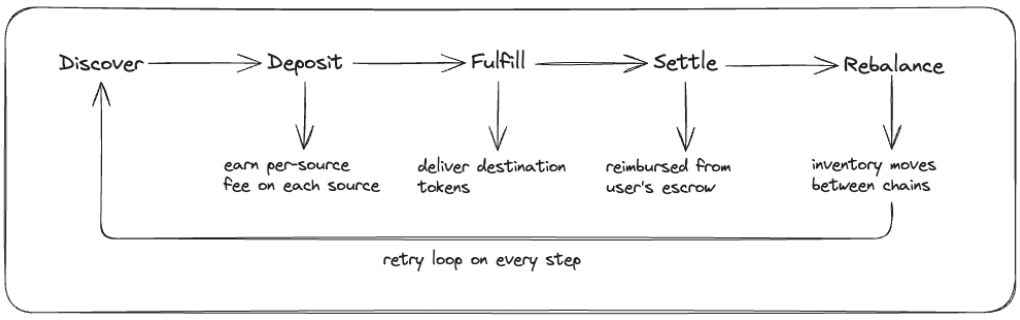


Fig. 5: Solver Lifecycle Phases

Discover. The poller calls the statekeeper's REST API `GET /rffs?status=created` and `GET /rffs?status=deposited`. Both states are queried because a user can self-deposit (the native-token case where the user's signature does not authorise a

contract `deposit` and they call the Vault directly). In that case, by the time the solver sees the RFF, it may already be in `Deposited` and the solver skips straight to fulfilment.

Deposit. For each source leg of each new RFF, the depositor evaluates profitability (gas vs. fee) before committing to submit. The pricing inputs are a gas estimate from a Tenderly-backed simulator and a token price from a CoinGecko client, both denominated in micro-dollars to avoid floating-point drift across the trip through the profitability calculator. If profitable, the depositor encodes a `Vault.deposit(request, signature, chainIndex)` call, hands it to the TxSubmitter, and marks the deposit as submitted in the local `RffStore`. On confirmation (the `Deposited` event arriving via the solver's own indexer), the solver collects the `SourcePair.fee` in the source token.

Fulfil. Once every source leg is confirmed *by the solver's own indexer*, the fulfiller re-runs profitability for the destination call (gas vs. the sum of source fees), encodes `Vault.fulfil(...)`, and submits. The destination Vault transfers the destination tokens to the user's `recipient_address`. The Vault emits a `Fulfilment` event whose `solver` field binds the fulfilling solver into the on-chain record.

Settle. This phase is passive from the solver's side. The statekeeper's STF, processing the destination `Fulfilment` event, generates one `SettlementInstruction` per source leg (see section 5.4). Each instruction names the solver as the recipient of the escrowed source funds. Validators sign the settlement data and submit `Vault.settle(...)` on each source chain. The solver receives reimbursement equal to the user's `SourcePair.value` on each source, plus the fees it already earned at deposit time.

Rebalance. Periodically (default 900s), the rebalancer compares each chain's balance against a target, and bridges surplus inventory toward chains running thin. It uses Stargate or Across with ordered provider failback, `PreSubmit` errors trigger fallback, `PostSubmit` errors stop. Rebalancing is internal solver bookkeeping and not a protocol mechanism; see `docs/REBALANCER.md` for the operational detail.

5.3.3 Profitability: Retry, Don't Reject

The profitability check sits in front of every deposit and fulfil submission. It is typed as a `Result<DepositProfit, DepositProfitError>` with four explicit failure branches:

- `NonceAlreadyUsed`, the on-chain nonce was already consumed (probably by an earlier attempt the indexer hasn't caught up with). Wait for the event.
- `GasEstimation(_)`, the gas oracle failed. Probably transient.
- `Pricing(_)`, the token price lookup failed. Almost always transient.
- `NotProfitable { gas, fee }`, the math worked, the answer is no.

Crucially, **none of these are hard rejections.** The depositor's response to every branch is the same: log, emit a metric, and move on. The same RFF is re-evaluated on the next tick, and the next, until either it becomes profitable or it expires. Gas markets are bursty; what is unprofitable at tick `t` is often profitable at `t + 30s`. Hard-rejecting at the first unprofitable evaluation would create user-visible failures for situations that resolve themselves in under a minute.

The cost of this design is real. RFFs have no fulfilment-time SLA; a user whose chosen expiry is 5 minutes and whose chosen fee is too thin to ever clear the gas cost will simply expire without anyone telling them why. The protocol pushes this cost onto the user's `expiry` choice, and onto whatever quoting layer sits in front of the user (the middleware can refuse to surface RFFs that are unlikely to clear at current gas). The benefit is that the protocol does not need a separate "RFF rejected" surface, and solvers do not need to coordinate on whose responsibility it is to reject. Every solver evaluates independently, every tick.

5.3.4 No Bonds, No Slashing

This is the most distinctive economic position in Nexus V2 relative to CoW Swap, UniswapX, and similar intent protocols. The solver posts no bond and faces no slashing. It is not registered with the protocol in any sense beyond knowing the statekeeper's REST endpoint and holding addresses funded to call the Vault contracts. There is no whitelist and no exit queue.

The solver's risk is bounded to its own capital. On the source side, the deposit escrows the *user's* funds, the solver only pays gas to submit it and earns the `SourcePair.fee`. The solver fronts its own capital on the destination side. On the destination side, it delivers the user's tokens and pays gas. If the validator set never signs and submits settlement, the solver has already delivered on the destination and is never reimbursed, it loses the fulfilled amount and the gas on every leg. There is no recourse mechanism in the protocol itself: the solver carries this risk in exchange for the per-source fees.

The mitigations that make this acceptable are protocol-level, not capital-at-risk:

1) **Independent deposit verification.** Before fulfilment, the solver consults its own indexer for the `Deposited` events. It does not rely on the statekeeper having sequenced them.
2) **Profitability gating.** The solver only proceeds when current gas plus historical fees make the leg net-positive.

3) **Validator-signed settlement.** The actor that authorises the solver's reimbursement (the validator set) is separate from the actor that orders intents (the statekeeper). A compromised statekeeper cannot drain solver capital.

The tradeoff against the bonded/slashed model is explicit. Without bonds, the protocol has no disincentive against a misbehaving solver beyond "they will lose their own money trying." For a protocol where solvers are professional market makers selecting profitable RFFs, this is sufficient. For a protocol that wanted to prevent, say, a solver from front-running other solvers' fulfilments or from collusion-pricing, bonds would be a tool, and Nexus V2 deliberately punts on those problems in exchange for simpler solver onboarding and no liquidation infrastructure. The closing section returns to this tradeoff.

A second consequence of the no-bonds model is that solver competition is currently first-come-first-served polling. Whichever solver's poller picks up an RFF and successfully submits the deposit first earns the fee. Solvers that lose a race waste a gas estimate and move on. An auction mechanism is conceivable; it is not the current design.

5.4 State Transition and Settlement Instructions

The statekeeper's central object is a state transition function (STF), a pure, deterministic function that takes a block of transactions and a pre-state map and returns a state diff, a header, and any side-effect instructions for validators to execute:

```
fn execute_block(
    transactions: &[Transaction],
    pre_state:    &HashMap<H256, StateLeaf>,
    /* ... protocol version, block context ... */
) -> Result<StfResult, StfError>;
```

The signature is unsurprising; what matters is what it omits. The STF performs no I/O. It does not query a database, fetch from an RPC, sample a clock, or generate randomness. Everything it needs is in the transaction stream and the pre-state map. This is a deliberate choice with two payoffs.

The first payoff is **testability**: state transitions are unit-testable as plain function calls with no infrastructure.

The second payoff is **verifiability by replay**. The validator does not need to trust the statekeeper. Given the same transactions (which it reads from Avail) and the same pre-state (which it derives from the previous block), the validator re-runs the STF and arrives at the same state root. If it does not, the statekeeper has diverged, and the validator refuses to finalise the header. The STF's purity is what makes this re-execution match bit-for-bit; any I/O would open a window for non-determinism.

5.4.1 Transaction Types

The STF accepts three transaction types:

Variant	Source	Effect
`CreateRff`	User, via API mempool	Insert a new RFF leaf with status `Created`
`ApplyNewBlockEvent`	Chain indexer, via block-events stream	Apply a block's `Deposited` / `Fulfilment` events; advance RFF state
`UpdateChainVaultLeaf`	Admin (Ed25519-signed)	Register or update a vault address for a chain

A fourth case, `ExpireRff`, is generated not by the STF but by the sequencer. At each tick the sequencer proposes expiries for RFFs whose expiry has passed *on every relevant chain*; the STF then validates each against on-chain block timestamps (not wall-clock, so the STF stays pure) and applies it. This is the only "transaction" not exposed to external submitters, and it is the mechanism by which the four-state lifecycle reaches its terminal `Expired` from outside the chain-event stream.

`UpdateChainVaultLeaf` carries an Ed25519 signature from a registered admin key. This is the protocol's only admin operation: registering new chains, rotating vault addresses, updating chain-level configuration. Routine state transitions do not require any admin authority.

5.4.2 Settlement Instructions

When the STF processes a destination chain's `Fulfilment` event for an RFF in `Deposited` status, it transitions the RFF to `Fulfilled` and, in the same step, emits a sequence of `SettlementInstructions`, one per source leg:

```
struct SettlementInstruction {
```

```
    universe:         Universe,
    chain_id:         u64,
    vault_address:    H256,  // source vault, resolved at RFF creation
    contract_address: H256,  // source token
    amount:           U256,  // SourcePair.value
    recipient:        H256,  // solver address (from Fulfilment event)
}
```

Refund instructions (emitted on `Expired` transitions for any source that had a confirmed deposit) follow the same shape with `recipient = parties[0].address`, the user.

These instructions are embedded in the block header. They are not stored as RFF state. The header carries them because the header is the validator's input: a validator that reads a finalised header has, in one place, the list of `(chain, vault, token, amount, recipient)` tuples it needs to sign and submit.

The design choice to put settlement here, rather than as an RFF status, has three implications.

Deterministic re-derivation from DA. A validator joining the network reads batches from Avail, re-executes the STF, and arrives at identical settlement instructions without consulting any side channel. The "what should be settled" question is fully answered by data availability. No validator needs to ask any other validator (or the statekeeper) for instructions; the canonical source is shared.

Partial and retryable settlement without state-machine complexity. A settlement that fails on one source chain (RPC outage, momentary lack of validator-side gas, transient revert) can be retried independently of all other settlements for the same RFF. The RFF's own state (`Fulfilled`) does not depend on whether settlement has happened. If the protocol embedded settlement in the RFF lifecycle, a partial failure on one of three sources would require either a fan-in state (`PartiallySettled`) or a per-source sub-state machine. By emitting instructions instead, the protocol gets idempotent retry for free; the validator simply re-attempts unsettled instructions on the next pass.

User experience does not couple to settlement liveness. The user's view is the four-state lifecycle. Once the RFF is `Fulfilled`, the user has been paid. Settlement of the solver is bookkeeping the user never sees. This decoupling matters for the protocol's UX promise: "your bridge worked" is a binary the user can read directly off the RFF state, even if validator-side reimbursement is mid-retry.

5.4.3 Storage

The statekeeper persists state across two stores. A Jellyfish Merkle Tree (kept in PostgreSQL in the default deployment, so the entire node state can be snapshotted with `pg_dump` for fast sync (a RocksDB backend exists as a feature-gated alternative)) holds the canonical key-value state (every RFF leaf, every chain registry leaf) keyed by `StateKey` and producing a state root that the validator verifies. PostgreSQL also holds everything that benefits from rich queries: the API mempool (`transactions` table), the chain-event mempool (`block_events` table), finalised headers, executed RFFs, and the Avail batch ledger.

The split is deliberate. The JMT is structured for two consumers: state root computation (Merkle proofs of inclusion) and a future zkVM execution (the same STF, the same state tree, proven). PostgreSQL is structured for the consumers that don't need cryptographic state proofs but do need indexed access (a user looking up RFFs by sender address, a validator looking up the next unfinalised batch). One store would not satisfy both.

For the detailed table layout, see `docs/ARCHITECTURE.md`.

5.5 Sequencing and Avail DA

The sequencer is a single 5-second tick that merges two streams and emits one batch per N blocks. The streams are the API mempool (RFF creations and admin transactions submitted via REST) and the chain-event mempool (indexer-streamed events from every Vault contract, written to PostgreSQL's `block_events` table). On each tick the sequencer drains some prefix of each stream, executes the resulting block through the STF, commits the header and state diff to storage, and adds the block to the current batch buffer.

A separate batcher loop turns buffered blocks into a `SignedBatch`:

```
struct SignedBatch {
    batch_id:            u64,
    blocks:              Vec<Vec<Transaction>>,  // one inner Vec per block
    sequencer_signature: Vec<u8>,                // Ed25519 over the canonical encoding
```

```
}
```

The batch is Borsh-serialised (Borsh is canonical and deterministic, which matters for signatures), signed with the sequencer's Ed25519 key, persisted to PostgreSQL with status `Pending`, and submitted to Avail with exponential backoff. On Avail inclusion the row moves to `Submitted`; once validators confirm the batch is fetchable and re-executed, the row moves to `Finalized`.

Two invariants are worth stating explicitly.

Batches strictly succeed each other by `batch_id`. A validator that has finalised batch `n` will not consider batch `n+1` until it has both batch `n` and batch `n+1` from Avail. The submitter writes batches in order; the validator reads them in order; the ordering is total, not merely partial.

A batch is one or more blocks. Block height and batch id are not the same number. The protocol uses blocks for STF-level granularity (so that one block can fit in one sequencer tick, and so headers can be issued frequently for indexer consumers) and batches for DA-level amortisation (so that each Avail submission carries enough data to be worth the inclusion fee). This separation lets the sequencer tune block frequency and batching policy independently. See `docs/ARCHITECTURE.md` for the precise block-to-batch packing flow.

5.5.1 Why Avail

Three properties of Avail justify its selection as Nexus V2's ordering and DA layer.

- **Data availability with cheap inclusion.** Avail's data-availability scheme makes batch inclusion economical at the rate Nexus needs (one batch every several sequencer ticks, carrying chain-event-frequency data, solvers and indexers want low latency, which inflates batch frequency). Posting equivalent data to Ethereum calldata at the same cadence would not be economically viable for a protocol of this throughput.
- **Canonical ordering by inclusion.** Avail's consensus provides a total order over included data blobs. Nexus inherits this order: batch `n` is whatever Avail says was included first, period. The sequencer does not need a separate consensus for ordering batches relative to each other; the inclusion order *is* the canonical order.
- **Recoverability from DA alone.** A validator (or anyone else) reading the full Avail history can reconstruct every Nexus state. There is no off-chain side channel required to follow the protocol. This is the foundation of the trust separation in section 5.8: the statekeeper is trusted for liveness (its data must reach Avail) and not for state correctness (anyone with Avail can re-derive and check).

Alternatives the protocol explicitly does not take:

- **A centralised sequencer with no DA layer.** Cheaper and simpler, but loses recoverability. A sequencer outage becomes a state loss; validators cannot independently re-derive.
- **Ethereum calldata.** Higher trust assumption (Ethereum's), but the cost at Nexus's data rate is prohibitive.
- **EigenDA or Celestia.** Plausible alternatives; the protocol does not preclude them. The choice of Avail is a current-generation decision, not a foundational constraint.
- **A shared rollup sequencer.** Couples Nexus to a specific L2 stack and inherits its sequencer's failure modes. Nexus prefers to own its sequencer abstraction.

The Ed25519 signature on each batch is what lets validators authenticate batches as "from the registered Nexus sequencer." The sequencer's pubkey is part of the genesis configuration. A validator that reads a batch from Avail and finds an unrecognised signature simply discards it. This makes Avail's role narrower than it could be: Avail proves *inclusion in some order*, the signature proves *this is a Nexus batch*. Neither alone is sufficient.

Statekeeper is trusted for	Falls back to
Including user RFFs in the mempool	(no fallback, censorship resistance is a roadmap item)
Producing canonically-ordered batches	Avail's ordering of included data
Signing batches with the registered key	Validator key-pinning at genesis
Computing correct state transitions	Validator re-execution detects divergence

5.6 Validation and Settlement Execution

A validator does two things. It finalises headers by re-executing the STF against Avail-sourced batches, and it authorises settlement by signing instructions and submitting them as `Vault.settle(...)` calls on source chains. The two responsibilities sit behind a common signer abstraction, which makes the validator the place where the protocol's signing and trust separation are concrete.

5.6.1 Header Finalisation

The validator reads batches from Avail in order. For each batch it verifies the sequencer's Ed25519 signature against the genesis-registered pubkey; if the signature is invalid, the batch is dropped. It then re-executes each block in the batch through the same STF the sequencer ran, producing its own state root and comparing against the header the sequencer committed. If the roots agree, the validator persists the header as finalised; if they disagree, the validator refuses to advance and surfaces the divergence.

Re-execution alone is not sufficient, because the chain events in a batch are inputs the sequencer supplies, replaying them reproduces the sequencer's state root whether or not those events ever happened on-chain. So before a batch's events are accepted, each validator independently verifies them against its own chain indexer: every `Deposited` / `Fulfilment` event in the batch must match an event the validator itself indexed at that block, and any block heights the sequencer skipped must be verifiably empty (`LocalEventVerifier`). A validator whose indexer has not yet reached the relevant height defers the batch rather than trusting it. This is what closes the gap a pure state-root check leaves open: a sequencer that fabricates or omits a chain event is caught at event verification, not at re-execution.

Header finalisation is the protocol's safety check. It is what makes the statekeeper untrusted with state correctness. The cost is a re-execution overhead, the validator pays compute equivalent to the sequencer's, and the gain is that no single statekeeper can corrupt the state without being caught by every validator.

5.6.2 Settlement Signing

Each finalised header carries the `SettlementInstruction`s the STF emitted. The validator's settlement path converts these into a `SettleData`, computes a canonical signing hash, signs it, and submits `Vault.settle(settle_data, signature)` on the relevant source chain.

The signing hash is constructed as:

1) Convert `SettleData` to its Solidity-boundary equivalent `VaultSettleData`.
2) ABI-encode the struct.
3) `keccak256` the encoded bytes.
4) Apply the standard EIP-191 envelope: `keccak256("19Ethereum Signed Message:32" || hash)`.

The signature output is 65 bytes in `[r | s | v]` format, with `s` normalised to low-s per EIP-2 to prevent malleability. The Vault contract verifies the same construction and rejects signatures that fail either canonicalisation.

The validator exposes settlement signing through a trait:

```
trait SettlementSigner: Send + Sync {
    async fn sign_prehash(&self, prehash: &H256)
        -> Result<EcdsaSignature, SignerError>;
    fn address(&self) -> Address;
    fn mode(&self) -> SigningMode;  // Local or Mpc
}
```

Two implementations exist:

- **`LocalSigner`** wraps a `PrivateKeySigner` and signs locally with a single private key. Appropriate for single-validator deployments and tests.
- **`MpcSigner`** performs DKLS23 threshold signing over a WebSocket relay (`msg-relay-svc`). Appropriate for production validator sets where no single party should be able to authorise settlement alone.

The mode is a config flag on the validator, not a protocol-level branch. The Vault contract sees a 65-byte ECDSA signature either way; whether that signature came from one private key or from a `t`-of-`n` party set is invisible at the contract layer. This is intentional. The protocol's safety story does not depend on which signer is in use, it depends on the validator set's collective honesty, and the choice of signing backend is an operational decision validators make. For the full MPC operational manual (ceremonies, keyshares, quorum management, relay topology), see `docs/MPC.md`.

5.6.3 Hot/Cold Separation: Settlement Authority vs. Gas Payer

A validator that submits `Vault.settle(...)` needs two distinct keys, and the protocol's design keeps them separated.

The **settlement authority** is the key (local or MPC) whose address the Vault contract recognises as authorised to settle. Compromise of this key allows forged settlements. It should be cold, hardware-backed, or threshold, ideally not present in the same process as anything network-facing.

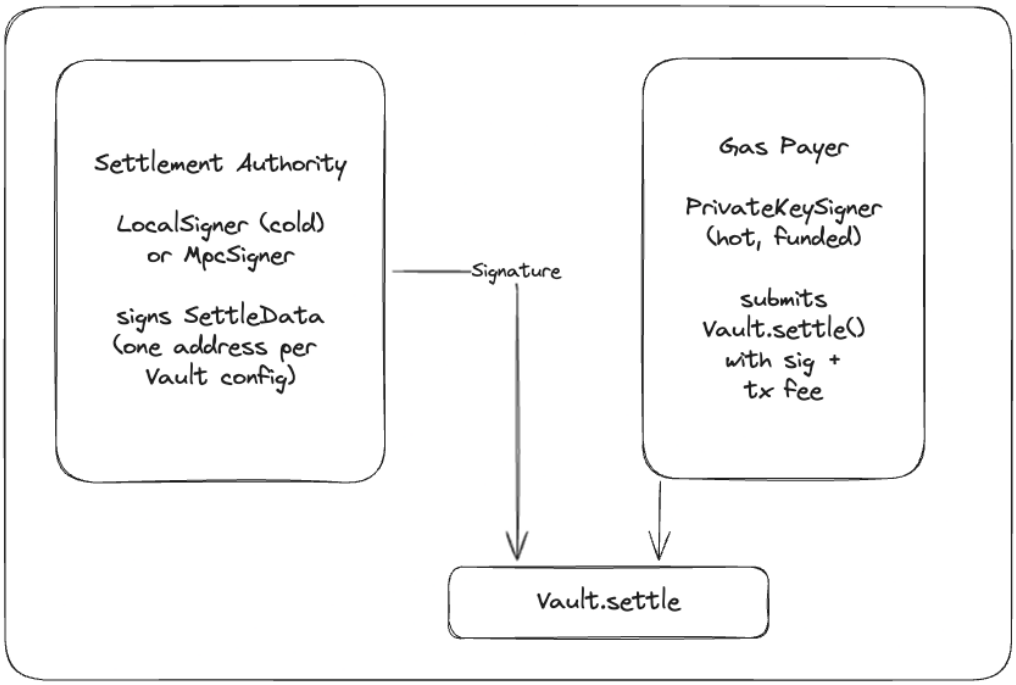


Fig. 6: Hot/Cold Key Separation for Settlement

The **gas payer** is a separate EOA holding native tokens to pay for transaction inclusion. It signs *the transaction wrapper* (an EVM transaction containing the settle call) but not the inner `SettleData`. Compromise of this key allows draining the gas wallet, but cannot forge settlements; the Vault rejects any settle call whose embedded ECDSA signature does not match the settlement authority's address.

This separation is what makes safe key management possible. A validator can keep the settlement key in a remote signer (or as MPC shares across machines that never connect to chain RPCs) while letting a hot, online process hold the gas payer key and handle the operational chore of resubmission, fee bumping, and nonce management. The protocol cannot enforce that validators set this up correctly, but the abstraction makes "do this right" a small operational step rather than a structural change.

5.7 Client Surface: The Middleware Layer

Nothing described so far is what a client application actually talks to. The statekeeper exposes a REST API, but in production deployments the SDK connects to a stateless **middleware** gateway that wraps the protocol with a set of client-facing affordances. The middleware is not part of the protocol, it can be replaced, bypassed, or horizontally scaled without touching the state machine, but it is what an external integrator works with day-to-day, and the protocol's "single-signature UX" claim leans on a specific piece of it.

The middleware is a TypeScript service built on Bun and Elysia. It is stateless: every request resolves against the deployment config, the statekeeper, and a handful of external services (CoinGecko, Tenderly, Ankr, aggregator APIs). It carries no consensus role and holds no protocol-relevant signing key. Its responsibilities fall into five buckets.

- **RFF proxying.** `POST /api/v1/rff`, `GET /api/v1/rff/:hash`, and `GET /api/v1/rffs` forward to the statekeeper's REST API with input validation, response shaping, and OpenTelemetry tracing. The middleware does not modify the signed request, the user's signature still covers the exact RFF that reaches the Vault contracts, but it canonicalises formatting (e.g., chain IDs as `uint256` hex, addresses as 32-byte padded) so SDK clients have one accepted shape.
- **Gasless approval sponsorship.** A WebSocket endpoint, `WS /api/v1/create-sponsored-approvals`, accepts user-signed approval payloads (EIP-2612 Permit or "EMT" / Execute Meta Transaction variants), submits the approval transactions on the user's behalf using a sponsor key, and streams per-chain results back. This is the mechanism behind the "single signature" claim of Section 5: the user signs the RFF and signs (off-chain, via permit) the approvals it depends on, the middleware pays gas to land the approvals on every relevant chain. The sponsor key is the middleware's one operationally sensitive secret; its compromise can drain the sponsor balance but cannot forge RFFs, settlements, or any user-relevant state.

- **Cross-chain fan-out reads.** `GET /api/v1/balance/EVM/:addr` aggregates a user's token balances across every supported chain using Ankr's multichain RPC with multicall fallback. `GET /api/v1/swap/:universe/:addr` does the same for swap-relevant tokens. `GET /api/v1/oracle` exposes a CoinGecko-backed price cache refreshed on a configurable cron (default 30s). These exist because an SDK that fanned out the same reads from a browser would face N RPC round-trips, N rate-limit budgets, and N CORS configurations; concentrating the fan-out in the middleware turns it into one HTTP request.
- **Secret-holding proxies.** `ALL /api/v1/proxy/{lifi,bebop,zerox}/*` is a pass-through to swap aggregator APIs with the aggregator's API key injected server-side. `POST /api/v1/gas/*` proxies Tenderly gas simulation with the Tenderly account credentials. Both let the SDK use paid third-party services without shipping API keys to clients. If a key is not configured the corresponding endpoints return `503` rather than failing in a less obvious way.
- **Deployment discovery.** `GET /deployment` returns the active network's chain metadata, vault addresses, token lists, explorer URLs, and asset logos. The SDK calls this on startup to bootstrap its view of the network; the middleware is therefore the single point where network-specific configuration is published to clients.

The middleware's role is best understood by what it deliberately *does not* do. It does not participate in sequencing, does not see settlement signing keys, does not hold validator MPC shares, and does not have any privilege in the Vault contracts beyond what the sponsor key buys. A protocol participant can build an alternative middleware, run integration directly against the statekeeper's REST API, or fork the existing service, and the rest of the protocol is unaffected. This is intentional: client-facing API design evolves faster than consensus rules, and the boundary lets the middleware iterate without the protocol team coordinating upgrades across validators.

Two design notes worth surfacing for integrators. First, the middleware is stateless and horizontally scalable; the infrastructure plan calls for a load-balanced cluster of middleware nodes in front of one statekeeper. Second, the middleware is instrumented end-to-end with OpenTelemetry (HTTP and WebSocket spans, log correlation via `trace_id` / `span_id`), so an operator can attribute latency to the RFF proxy, the aggregator pass-through, or the upstream statekeeper without per-endpoint tracing work.

5.8 Trust Model

A consolidated map of who is trusted for what, what they cannot be trusted with, and what mitigates the failure if they are dishonest.

Actor	Trusted for	Not trusted for	Mitigation if violated
User	Signing their own RFF	Honest input (replay protection via per-user `nonce`)	Vault rejects invalid signatures and replayed nonces
Statekeeper	Liveness, mempool inclusion, canonical batch ordering	State correctness, settlement authorisation	Validators independently reindex chain events *and* re-execute the STF from Avail; a fabricated/omitted event or a bad state root $\rightarrow$ no finalisation, no settlement
Avail DA	Data availability, total ordering of included batches	Execution correctness	Avail's own consensus and DA guarantees
Validator set	Settlement signing, header finalisation	Single-validator liveness	Threshold signing (MPC) or external redundancy
MPC relay	Honest message transport between parties	Confidentiality of key shares (cryptographically)	Authenticated transport is on the roadmap; see `docs/MPC.md`
Middleware	Client-facing UX (RFF formatting, balance reads, sponsorship of approvals)	Anything protocol-level	Stateless and replaceable; the statekeeper REST API is the protocol's actual entry point. Sponsor key compromise drains sponsor balance only
Solver	Self-interest only	Anything else	No protocol trust; solver risks only its own capital

Three of these rows are worth a sentence of unpacking.

The statekeeper is not trusted with state. This is the protocol's strongest claim. A compromised or buggy statekeeper that signs a divergent batch is detected at the validator's re-execution step, the state root mismatch refuses finalisation, and settlements derived from the divergent batch are never signed. The statekeeper can stall the protocol but cannot corrupt it.

Validators are the actor that solvers depend on for liveness. Once a solver has fronted capital for a deposit and delivered fulfilment, its reimbursement depends on the validator set actually signing and submitting settlement. A failure mode of "everyone's RFFs are getting fulfilled but solvers stop getting paid" is the protocol's most acute liveness pressure. Validator design (threshold count, redundancy, gas-wallet funding monitoring) is the place the protocol pushes most of its operational risk.

The MPC relay is currently in the trusted path for transport. It does not see plaintext signatures or key shares (those are cryptographic constructs the relay handles as opaque blobs), but a malicious relay could refuse to route messages between parties, blocking signing. This is documented as a known limitation; authenticated transport is on the roadmap.

For the operational side of the trust model, how validators run, how MPC ceremonies are conducted, what the relay does and does not handle, see `docs/MPC.md` and the validator section of `docs/ARCHITECTURE.md`.

5.9 Design Discussion and Open Questions

Six choices distinguish Nexus V2 from adjacent designs. Each is a tradeoff, not a universal answer.

- **Solver-fronted capital, not lock-and-mint.** The user receives the source chain's native asset on the destination, not a wrapped representation. This is what makes the protocol a chain-abstraction layer rather than a bridge. The cost is that solvers carry the working capital; the protocol cannot run with zero solver liquidity. The benefit is no wrapped fragmentation and no claim step.
- **Settlement as instruction, not state.** Settlement instructions live in the block header; the RFF state machine has only four states. This is what makes settlement retryable, idempotent, and decoupled from the user's view of "did my bridge work?". The cost is one extra concept in the protocol description; the benefit is operational simplicity and a cleaner user-facing state.
- **No bonds, no slashing.** Solvers carry their own capital risk and have no posted bond. This is what makes solver onboarding frictionless and removes a class of liquidation infrastructure. The cost is that the protocol has no in-protocol disincentive against solver misbehaviour beyond the solver losing its own money, adequate for professional market makers, weak against adversarial solver populations. Auction mechanisms and bonded competition are not precluded by the design; they are not the current design.
- **Avail DA, not centralised sequencer, ETH calldata, or shared rollup sequencer.** Avail's combination of cheap inclusion, total ordering, and DA-only recoverability hits the operating point the protocol needs at its data rate. The choice is not foundational; EigenDA or Celestia are conceivable alternatives. The constraint is "must be a true DA layer," not "must be Avail."
- **Pure STF.** Deterministic, I/O-free, replayable. The cost is the discipline required to keep it that way; the benefit is testability today and zkVM compatibility tomorrow.
- **Hot/cold signer separation.** Settlement authority is decoupled from gas payment. A protocol choice, the Vault contract treats them separately, that lets validators arrange their key management with as much (or as little) operational care as their risk tolerance demands.

5.9.1 Open Questions

Solver liveness pressure on settlement. If validators slow down or fail to sign settlements, solvers continue fronting capital but stop being reimbursed. Validator-set health is therefore the protocol's most operationally sensitive surface. Threshold signing makes this less acute but does not eliminate it.

MPC relay authentication. The relay sits in the message-transport path between MPC parties. Today it is in the trusted set. An authenticated-transport design that removes it from the trusted set is on the roadmap and is the most significant in-flight security improvement.

Cross-universe extension. The `Universe` enum is first-class in every data structure (RFF, settlement, vault address). The current implementation is EVM-only, non-EVM sources are skipped with a warning at instruction generation time. Extending the protocol to a non-EVM universe is a per-universe matter of implementing the `UniverseOps` trait (signing hash, address conversion, vault encoding) plus an indexer for that chain family. The protocol does not preclude this; it does not yet deliver on it.

Solver competition design. First-come-first-served polling is the current solver-selection mechanism. An auction (sealed-bid, batch, or otherwise) is a possible direction, with attendant complexity in mempool design and the question of who runs the auction. The protocol punts; the surface is open.

References

[1] M. Al-Bassam, A. Sonnino, and V. Buterin, “Fraud and data availability proofs: Maximising light client security and scaling blockchains with dishonest majorities,” 2019.

[2] A. Kate, G. M. Zaverucha, and I. Goldberg, “Constant-size commitments to polynomials and their applications,” in *Advances in Cryptology - ASIACRYPT 2010*, M. Abe, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 177–194.

[3] V. Buterin, *2D data availability with Kate commitments*, 2020 (accessed November 24, 2020). [Online]. Available: https://ethresear.ch/t/2d-data-availability-with-kate-commitments/8081

[4] Polkadot, *Polkadot Consensus*, 2020 (accessed December 18, 2020). [Online]. Available: https://wiki.polkadot.network/docs/en/learn-consensus

[5] H. K. Alper, *BABE*, 2020 (accessed December 18, 2020). [Online]. Available: https://research.web3.foundation/en/latest/polkadot/block-production/Babe.html

[6] A. Stewart and E. Kokoris-Kogia, “Grandpa: a byzantine finality gadget,” 2020.